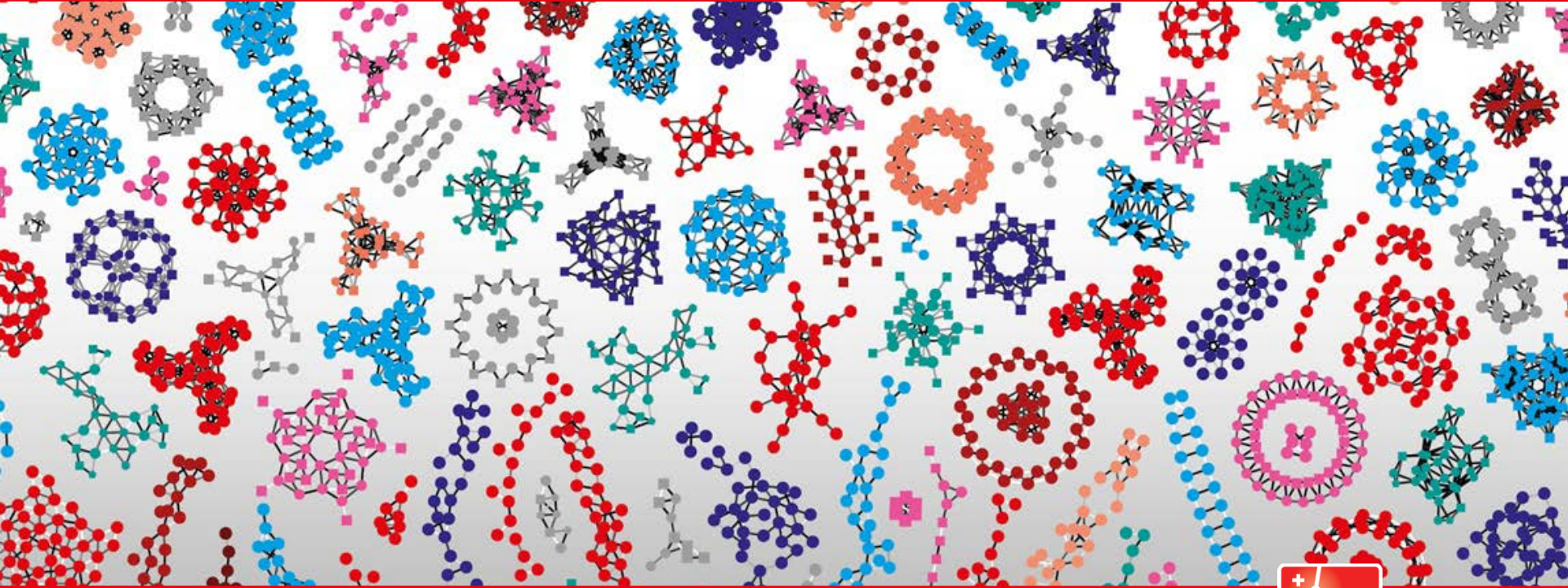




Swiss Institute of
Bioinformatics

R packaging 2026

Frédéric Schütz (Frederic.Schutz@sib.swiss)

**How can we reuse or
distribute R code ?**

Several ways to distribute R code

- Copy/paste snippets of code
- Source files
- Packages

« In R, the fundamental unit of shareable code is the package. A package bundles together code, data, documentation, and tests, and is easy to share with others. »

– Hadley Wickham



Contributed Packages

Available Packages

Currently, the CRAN package repository features **22578** available packages.

[Table of available packages, sorted by date of publication](#)

[Table of available packages, sorted by name](#)

[CRAN Task Views](#) aim to provide some guidance which packages on CRAN are relevant for tasks related to a certain topic. They provide tools to automatically install all packages from each view. Currently, 48 views are available.

Installation of Packages

Please type `help("INSTALL")` OR `help("install.packages")` in R for information on how to install packages from this repository. The manual [R Installation and Administration](#) (also contained in the R base sources) explains the process in detail.

Package Check Results

All packages are tested regularly on machines running [Debian GNU/Linux](#),

[CRAN](#)
[Mirrors](#)
[What's new?](#)
[Search](#)
[CRAN Team](#)

[About R](#)
[R Homepage](#)
[The R Journal](#)

[Software](#)
[R Sources](#)
[R Binaries](#)
[Packages](#)
[Task Views](#)
[Other](#)

Why create R packages ?

If you want to ...

- share code
 - organize code
 - version code
 - make your code easily available, even if only for yourself
-
- R packages are useful both for **external** distribution and for **internal** use

R packages are a fundamental tool
in order to make
your work with R **reproducible**

How to use R packages ?

Downloading, installing and loading an R package

From the console

```
# Package from CRAN
> install.packages("ABPS")
Installing package into '/usr/local/lib/R/site-library'
(as 'lib' is unspecified)
[...]
* DONE (ABPS)
> library(ABPS)
>
```

The output is sometimes more verbose

```
> library(Hmisc)
```

```
Loading required package: lattice
```

```
Loading required package: survival
```

```
Loading required package: Formula
```

```
Loading required package: ggplot2
```

```
Attaching package:  Hmisc
```

```
The following objects are masked from  package:base :
```

```
    format.pval, units
```

The output is sometimes more verbose

```
> library(tidyverse)
— Attaching packages — tidyverse 1.3.1 —
✓ ggplot2 3.3.6      ✓ purrr 0.3.4
✓ tibble 3.1.7       ✓ dplyr 1.0.9
✓ tidyr 1.2.0        ✓ stringr 1.4.0
✓ readr 2.1.2        ✓ forcats 0.5.1
— Conflicts — tidyverse_conflicts() —
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag() masks stats::lag()
> █
```

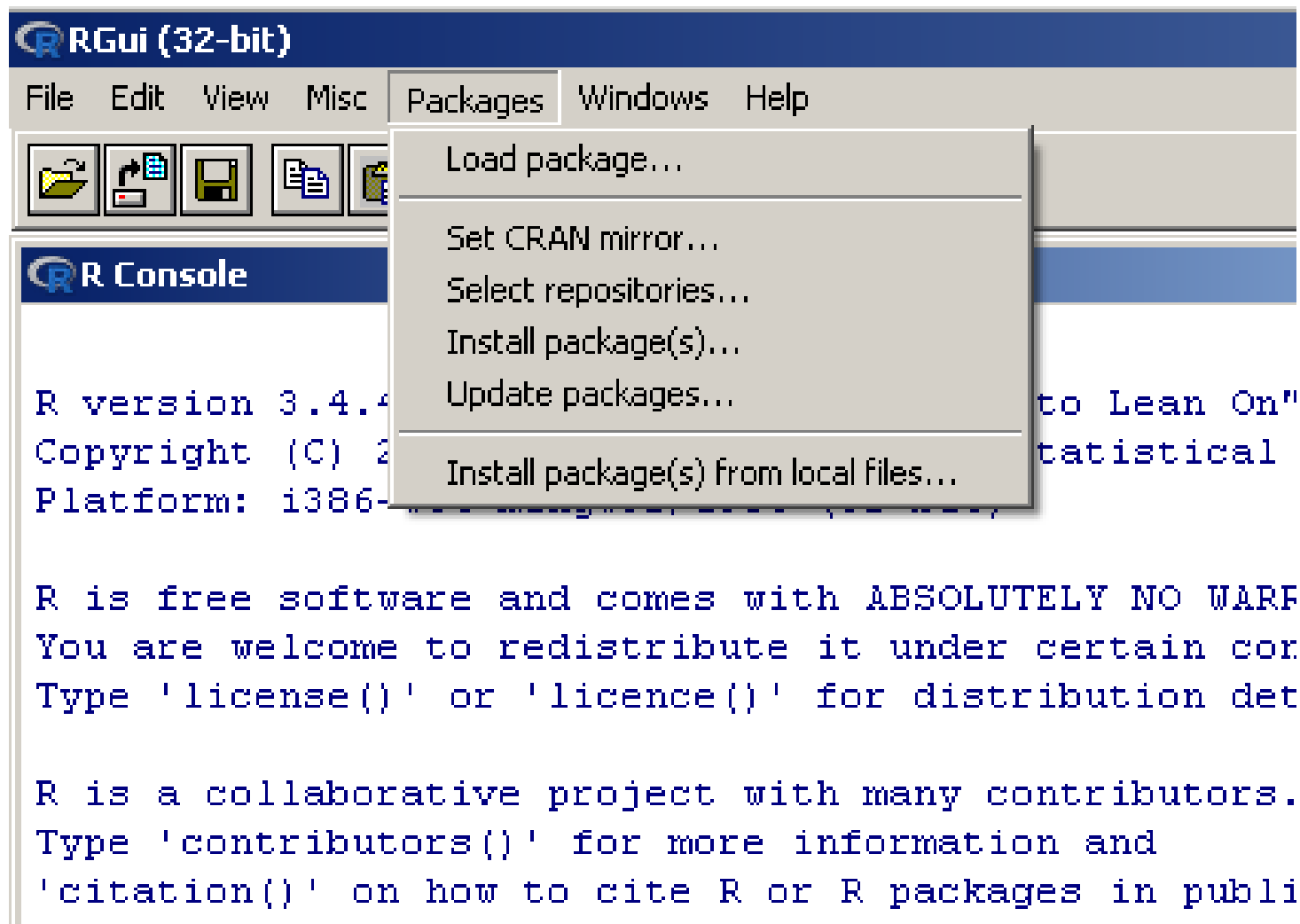
Installing a local R package

From the console

```
# Local package file
> install.packages("test_0.01.tar.gz")
Installing package into '/usr/local/lib/R/site-library'
(as 'lib' is unspecified)
inferring 'repos = NULL' from 'pkgs'
[...]
* DONE (test)
>
```

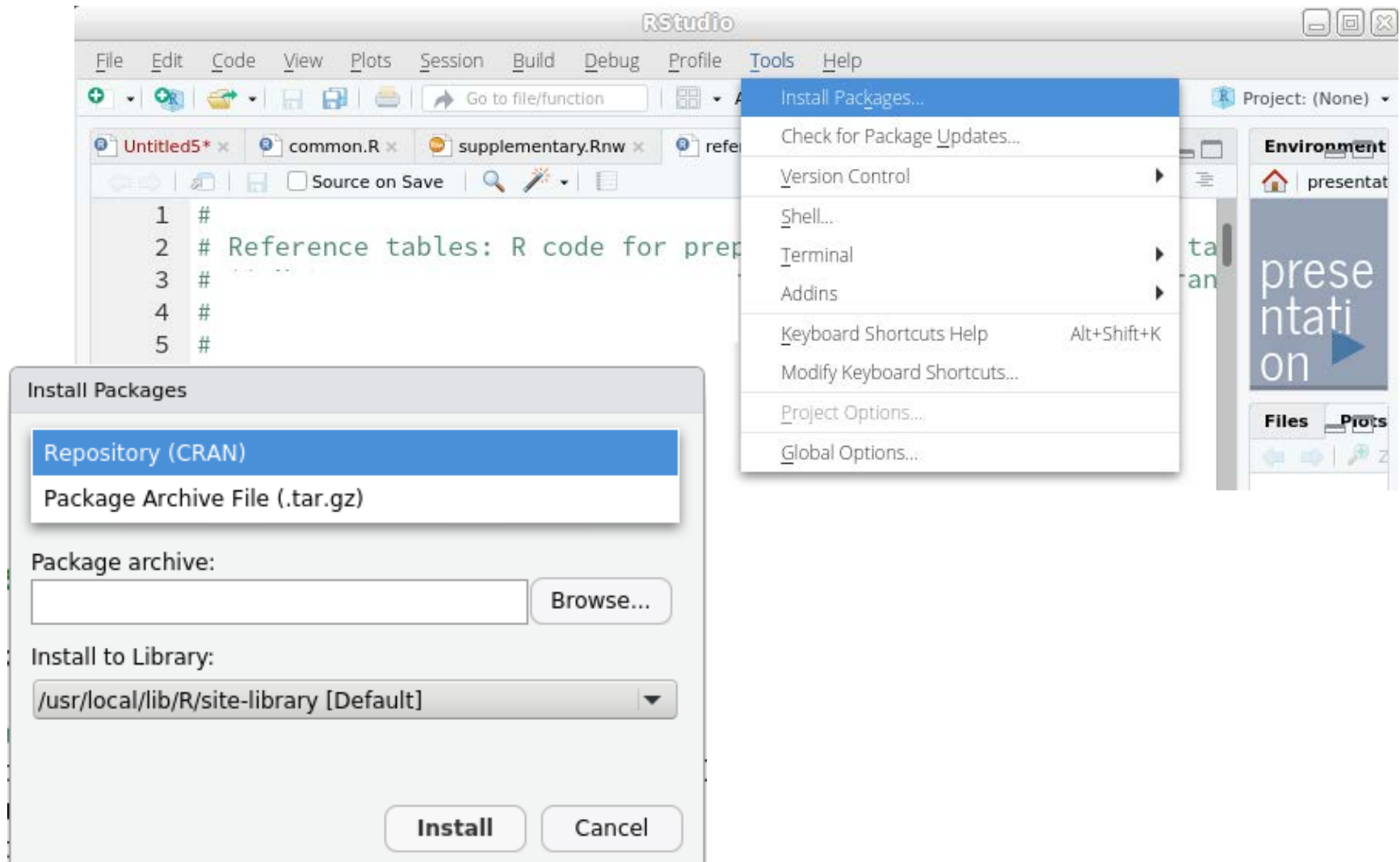
Downloading, installing and loading an R package

From the basic R GUI (Windows)



Downloading, installing and loading an R package

From RStudio



Note about the "lib" parameter

When calling `install.packages()`, you can specify where the new package should be installed, using the "lib" parameter.

You can install packages in system directories (if you have the right to do so), or in your home directory.

See:

- `.libPaths()`

- `.Library()`

How to create R packages

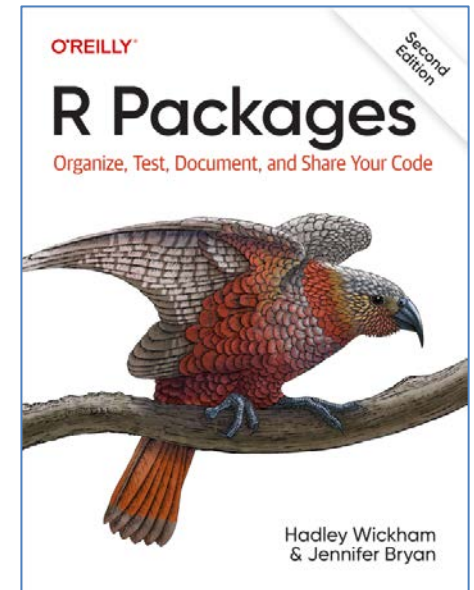
Caveats

- Creating a basic R package is **extremely** simple (as we will see).
- Creating an R package for your own use is relatively easy.
- Creating an R package that you can publish/distribute is much harder
- Creating an R package that you can publish in a repository (CRAN, github) will require you to write some very polished code.
It will likely take much longer than the time spent actually writing the R code inside the package.

Documentation

References

Hadley Wickham, Jennifer Bryan
*"R packages: Organize, Test,
Document, and Share your code"*
Print: 2nd ed, O'Reilly, June 2023
Up-to-date version online at:
<http://r-pkgs.org/>

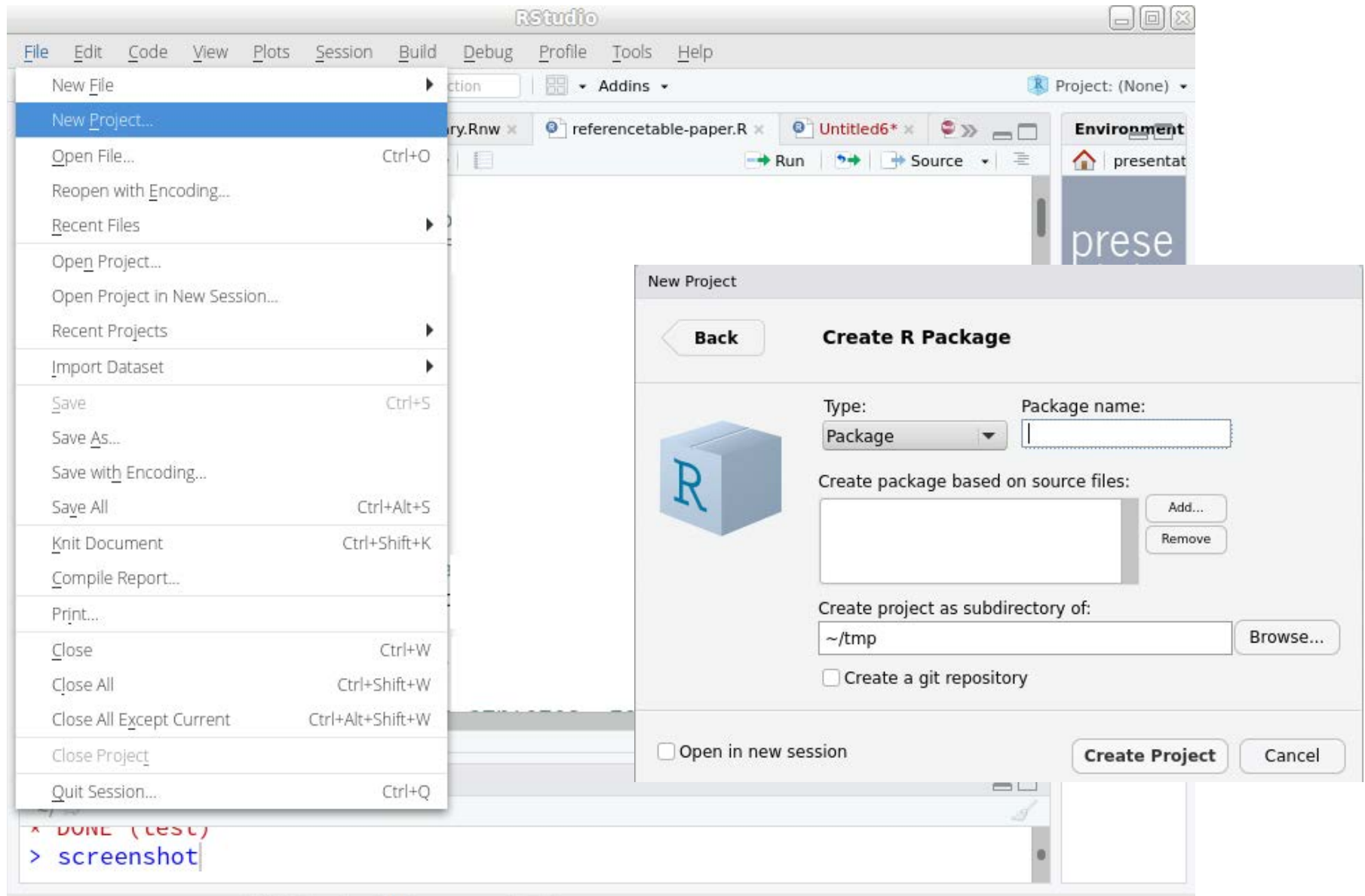


"Writing R Extensions"
<https://cran.r-project.org/doc/manuals/r-release/R-exts.html>

Prerequisites

R contains the bare minimum in order to build packages.

Highly recommended: RStudio





Highly recommended: the devtools package

devtools: Tools to Make Developing R Packages Easier

Collection of package development tools.

Version: 2.4.5

Depends: R (≥ 3.0.2), [usethis](#) (≥ 2.1.6)

Imports: [cli](#) (≥ 3.3.0), [desc](#) (≥ 1.4.1), [ellipsis](#) (≥ 0.3.2), [fs](#) (≥ 1.5.2), [lifecycle](#) (≥ 1.0.1), [memoise](#) (≥ 2.0.1), [miniUI](#) (≥ 0.1.1.1), [pkgbuild](#) (≥ 1.3.1), [pkgdown](#) (≥ 2.0.6), [pkgload](#) (≥ 1.3.0), [profvis](#) (≥ 0.3.7), [rcmdcheck](#) (≥ 1.4.0), [remotes](#) (≥ 2.4.2), [rlang](#) (≥ 1.0.4), [roxygen2](#) (≥ 7.2.1), [rversions](#) (≥ 2.1.1), [sessioninfo](#) (≥ 1.2.2), stats, [testthat](#) (≥ 3.1.5), tools, [urlchecker](#) (≥ 1.0.1), utils, [withr](#) (≥ 2.5.0)

Suggests: [BiocManager](#) (≥ 1.30.18), [callr](#) (≥ 3.7.1), [covr](#) (≥ 3.5.1), [curl](#) (≥ 4.3.2), [digest](#) (≥ 0.6.29), [DT](#) (≥ 0.23), [foghorn](#) (≥ 1.4.2), [gh](#) (≥ 1.3.0), [gmailr](#) (≥ 1.0.1), [httr](#) (≥ 1.4.3), [knitr](#) (≥ 1.39), [lintr](#) (≥ 3.0.0), [MASS](#), [mockery](#) (≥ 0.4.3), [pingr](#) (≥ 2.0.1), [rhub](#) (≥ 1.1.1), [rmarkdown](#) (≥ 2.14), [rstudioapi](#) (≥ 0.13), [spelling](#) (≥ 2.2)

Published: 2022-10-11

Author: Hadley Wickham [aut], Jim Hester [aut], Winston Chang [aut], Jennifer Bryan  [aut, cre], RStudio [cph, fnd]

Maintainer: Jennifer Bryan <jenny at rstudio.com>

BugReports: <https://github.com/r-lib/devtools/issues>

License: [MIT](#) + file [LICENSE](#)

URL: <https://devtools.r-lib.org/>, <https://github.com/r-lib/devtools>

NeedsCompilation: no

Language: en-US

Materials: [README](#) [NEWS](#)

CRAN checks: [devtools results](#)

You may also need:

- LaTeX for building manuals in PDF
- GNU software tools, if your package is compiled (for example, if it contains C code)

- **Linux:** depends on your distribution
for example, Ubuntu/Debian:
`sudo apt install r-base-dev texlive-full`
- **Mac:**
Install "Command Line Tools for XCode"
Install MacTeX
- **Windows**
Install the Rtools:
<https://stat.ethz.ch/CRAN/bin/windows/Rtools/>
Install TinyTeX: <https://yihui.org/tinytex/>
or Miktex: <http://miktex.org/download>

How to create a basic R package ?

What you need to create a basic "test" package by hand

test (directory), containing
DESCRIPTION (file)

The DESCRIPTION file should contain at least two lines:

```
Package: test  
Version: 1.0
```

**How to make an R package out
of this directory ?**

The R software can be used as a batch utility:

R CMD command ...

Examples:

R CMD INSTALL package

R CMD REMOVE package

R CMD BATCH script.R

R CMD build directory

Creating a source package from the command line

Go to the directory which contains the package directory (one directory up from the DESCRIPTION file)

Call the R batch command:

```
R CMD build DIRECTORY
```

Creating a source package from the command line

This is relatively easy to do when using Linux or Mac

Under Windows, it may be harder to call the program.
For example, the command may look like:

```
C:\PROGRA~1\R\R-3.4.4\bin\R.exe
```

```
CMD build directory
```

Example of package build

```
schutz@laptop:~/packages$ R CMD build test
* checking for file 'test/DESCRIPTION' ... OK
* preparing 'test':
* checking DESCRIPTION meta-information ... OK
* checking for LF line-endings in source and
  make files and shell scripts
* checking for empty or unneeded directories
* building 'test_0.01.tar.gz'

schutz@laptop:~/packages$ ls -l test_0.01.tar.gz
[...]
```

What you need to create a basic "test" package by hand

test (directory), containing
DESCRIPTION (file)

The DESCRIPTION file contains two lines:

```
Package: test  
Version: 1.0
```

**This is the bare minimum
required to create
an R package**

What you need to create a basic "test" package by hand

test (directory), containing
DESCRIPTION (file)

The DESCRIPTION file contains two lines:

```
Package: test  
Version: 1.0
```

**This is the bare minimum
required to create
an R package
... which is completely useless.**

*The most basic **useful** package*

test (directory), containing

DESCRIPTION (file)

NAMESPACE (file)

R (directory), containing

any_file.R (files), containing R functions

The DESCRIPTION file contains two lines:

Package: test

Version: 1.0

The NAMESPACE file indicates which functions are made available to users of the package.

Note about the package name

The name of the package is provided by the DESCRIPTION file.

The name of the directory containing the package file should have the same name, in order to make your life easier.

(You *could* choose any other name if you want, but you should really, really, not do this !)

The R directory

- The R directory contains .R files which are executed when building the package, thus creating the function objects
- The .R files must contain the assignment of functions:

```
myfunction <- function(...) {...}
```
- You are free to organize the files as you wish (one or more functions per file)

Content of the R directory

myfunctions_1.R

```
myfunction <- function(arguments) {  
  ...  
}  
  
plotdata <- function(data, add=TRUE) {  
  ...  
}
```

otherfunctions.R

```
analyze_data <- function(arguments) {  
  ...  
}  
  
pipeline <- function(data, add=TRUE) {  
  ...  
}
```

The NAMESPACE file

- Any function you define in the R directory is by default internal to the package only
- To make it available to the user, you need to **export** it explicitly.

You can export a function by adding a line

```
export(myfunction)
```

in the file `NAMESPACE` for each function you want to export.

R subdirectory

myfunctions_1.R

```
myfunction <- function(arguments) {...}  
plotdata <- function(data, add=TRUE) {...}
```

otherfunctions.R

```
analyze_data <- function(arguments) {...}  
pipeline <- function(data, add=TRUE) {...}  
  
internalfunction <- function(data) {...}
```

NAMESPACE

```
export(myfunction)  
export(plotdata)  
export(analyze_data)  
export(pipeline)
```

You can also use patterns to export multiple functions simultaneously:

```
exportPattern("^[:alpha:]+")
```

Using better tools

It is possible to build packages using the command line.

However, this can be cumbersome – especially if you use Windows.

RStudio allows you to manage packages (in the form of projects) in a more convenient way.

- Create the minimum R package, build it, install it and load it
- Do the same with a package that is actually useful: add an R function, and make sure that you can use it after loading the package.
- If you need help finding R functions to package, look for suggestion in the course folder.

Build again the same package as before, starting from an empty RStudio project.

Congratulations !

Going further...

The DESCRIPTION file

The DESCRIPTION file

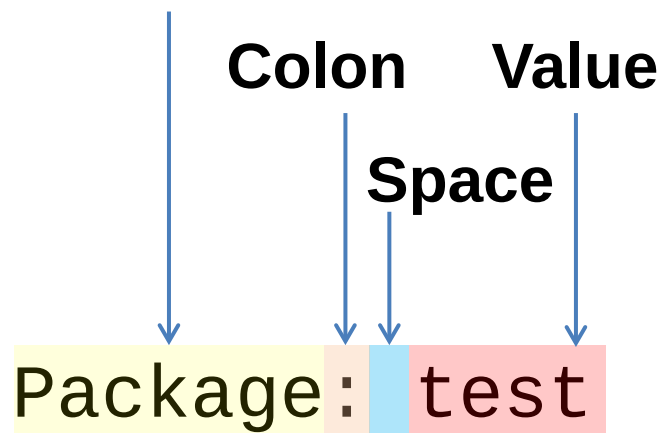
The DESCRIPTION file contains the metadata regarding the package.

The presence of this file is what makes a directory recognized as an R package.

Field name (ASCII)

Colon **Value**

Space



Package: test

Title: What the Package Does (one line)

Version: 0.1

Description: What the package does (one
paragraph). The description can span
several lines if needed.



Indentations (spaces or tabs) indicate multi-line fields

This file format is called "Debian Control File" (DCF)

See the help page for the `read.dcf()` function for more information.

DESCRIPTION File format

Field names are case-sensitive.

All those used by R are capitalized.

Some fields expect a boolean value.

You can use either "yes", "true", "no", "false"
(or a capitalized version of these)

Example

devtools: Tools to Make Developing R Packages Easier

Collection of package development tools.

Version: 2.4.5

Depends: R (≥ 3.0.2), [usethis](#) (≥ 2.1.6)

Imports: [cli](#) (≥ 3.3.0), [desc](#) (≥ 1.4.1), [ellipsis](#) (≥ 0.3.2), [fs](#) (≥ 1.5.2), [lifecycle](#) (≥ 1.0.1), [memoise](#) (≥ 2.0.1), [miniUI](#) (≥ 0.1.1.1), [pkgbuild](#) (≥ 1.3.1), [pkgdown](#) (≥ 2.0.6), [pkgload](#) (≥ 1.3.0), [profvis](#) (≥ 0.3.7), [rcmdcheck](#) (≥ 1.4.0), [remotes](#) (≥ 2.4.2), [rlang](#) (≥ 1.0.4), [roxygen2](#) (≥ 7.2.1), [rversions](#) (≥ 2.1.1), [sessioninfo](#) (≥ 1.2.2), stats, [testthat](#) (≥ 3.1.5), tools, [urlchecker](#) (≥ 1.0.1), utils, [withr](#) (≥ 2.5.0)

Suggests: [BiocManager](#) (≥ 1.30.18), [callr](#) (≥ 3.7.1), [covr](#) (≥ 3.5.1), [curl](#) (≥ 4.3.2), [digest](#) (≥ 0.6.29), [DT](#) (≥ 0.23), [foghorn](#) (≥ 1.4.2), [gh](#) (≥ 1.3.0), [gmailr](#) (≥ 1.0.1), [httr](#) (≥ 1.4.3), [knitr](#) (≥ 1.39), [lintr](#) (≥ 3.0.0), [MASS](#), [mockery](#) (≥ 0.4.3), [pingr](#) (≥ 2.0.1), [rhub](#) (≥ 1.1.1), [rmarkdown](#) (≥ 2.14), [rstudioapi](#) (≥ 0.13), [spelling](#) (≥ 2.2)

Published: 2022-10-11

Author: Hadley Wickham [aut], Jim Hester [aut], Winston Chang [aut], Jennifer Bryan  [aut, cre], RStudio [cph, fnd]

Maintainer: Jennifer Bryan <jenny at rstudio.com>

BugReports: <https://github.com/r-lib/devtools/issues>

License: [MIT](#) + file [LICENSE](#)

URL: <https://devtools.r-lib.org/>, <https://github.com/r-lib/devtools>

NeedsCompilation: no

Language: en-US

Materials: [README](#) [NEWS](#)

CRAN checks: [devtools results](#)

The 7 mandatory fields of a DESCRIPTION file



The 7 "mandatory" fields in the DESCRIPTION

**You need these
in order to *create*
the package**

Package: test

Version: 0.0.1

Title: What the Package Does (one line, title case)

Description: What the package does (one paragraph),
over one or more lines.

Author: John Smith

Maintainer: John Smith <john@smith.com>

License: What license is it under?

**Later *checks* of the package will *fail*
without these**

Each package must have at least one author and one maintainer (which may be the same person)

Author: people who have made significant contributions to the package

Maintainer: the person responsible for the package, the main contact

At least the maintainer should have an email address.

You can specify the authors in a more flexible way using the Authors@R field.

This field contains an executable R command, containing 4 fields for each author, e.g. :

```
Authors@R: person("First",  
                  "Last",  
                  email = "first.last@example.com",  
                  role = c("aut", "cre"))
```

The Author and Maintainer fields will be automatically generated from this field.

The most important roles:

- `cre`: the creator or maintainer, the person responsible for the package, main contact
- `aut`: authors (people who have made significant contributions to the package)
- `ctb`: contributors (people who have made smaller contributions)

(note: there are many other possible roles available, see <https://www.loc.gov/marc/relators/relaterm.html>)

You can specify several authors using a vector:

```
Authors@R: c(  
  person("Joe", "Developer", role = c("aut", "cre"),  
        email = "Joe.Developer@some.domain.net"),  
  person("A.", "User", role = "ctb",  
        email = "A.User@whereever.net")  
)
```

The package ***must*** specify under which conditions it can be reused and distributed.

... even if you do not intend to distribute it.

This is done by specifying a **License**.

A license is a contract, which allows an author to specify what a user is allowed to do (or not do) with software that he created.

R recognizes shortcuts for many common software licenses, such as:

- GNU General Public Licence
GPL - 2, GPL - 3, GPL (>=2)
- BSD or MIT licenses
MIT, BSD_2_clause
- Creative Commons licenses
CC BY-SA 4.0, CC BY 4.0
- No restrictions
Unlimited, CC0

R is centred on the concept of **free software**, which

- can be used by anyone without any limitation
- can be studied (e.g. you can read the code)
- can be modified
- can be redistributed (including the modifications)

R makes it easy to distribute your code as free software.

R itself is distributed under the GPL

If you want to submit your package to the CRAN repository, it must be distributed under a recognized free software license.

You can use a custom license by creating a file called LICENSE that contains the terms of your licenses and by specifying

```
License: file LICENSE
```

in the DESCRIPTION file.

You can use the license file to indicate that a package is proprietary (e.g. internal to your group) and should not be distributed.

However, you should **not** try to go further and write an actual license that you will distribute – use a standard license or ask a lawyer first.

If you need help choosing a license, please ask (during the class, during the breaks, or afterwards by email).

At least two (most often 3) integers, separated by either "." or "-".

The canonical form is "1.0-1".

This is used to check for dependencies (for example, for specifying minimum versions of software to use)

A version field does not represent a decimal number – the parts must be considered separately:

Version 0.9 < 0.75

0.9 > 0.7

Because 9 < 75

9 > 7

The 7 mandatory fields

Package: test

Version: 0.0.1

Title: What the Package Does (one line, title case)

Description: What the package does (one paragraph),
over one or more lines.

Author: John Smith

Maintainer: John Smith <john@smith.com>

License: What license is it under?

DESCRIPTION File format

The field names and the package name can only contain ASCII characters:

A-Z, a-z, 0-9, plus a few signs

Ideally, the *whole* file should be written using only ASCII characters.

DESCRIPTION File format

It is not always possible – e.g. if an author has accents in his/her name, we need more than ASCII.

In this case, the file must specify an encoding, e.g.

Encoding: UTF-8

Encoding ?

A (character) encoding links a character with the actual (binary) data written on disk or in memory.

For example, in ASCII:

Letter	ASCII Code	Binary
A	65	01000001
B	66	01000010
C	67	01000011
D	68	01000100

The same binary code may mean different things when using different encodings; the computer has to be told which one is correct.

This explains why you may sometimes see some strange characters when using characters that go beyond simple letters and numbers.

Nowadays, UTF-8 is by far the most common encoding and is almost certainly the right option to choose.

Encoding: UTF8 vs ISO-8559-1

Letter "é"	UTF8	ISO-8559-1
U+00E9 11000011 10101001	é	Ã©
é 11101111	?	é

Package: ABPS

Title: The Abnormal Blood Profile Score to Detect Blood Doping

Version: 0.3

Description: An implementation of the Abnormal Blood Profile Score (ABPS)

Imports: kernlab

License: GPL (≥ 2)

Encoding: UTF-8

Author: Frédéric Schütz [aut, cre], Alix Zollinger [aut]

Maintainer: Frédéric Schütz <schutz@mathgen.ch>

Saved as ISO-8559-1, displayed as UTF-8

Package: ABPS

Title: The Abnormal Blood Profile Score to Detect Blood Doping

Version: 0.3

Description: An implementation of the Abnormal Blood Profile Score (ABPS)

Imports: kernlab

License: GPL (>= 2)

Encoding: UTF-8

Author: Fr d ric Schutz [aut, cre], Alix Zollinger [aut]

Maintainer: Fr d ric Schutz <schutz@mathgen.ch>

Saved as UTF-8, displayed as ISO-8559-1

Package: ABPS

Title: The Abnormal Blood Profile Score to Detect Blood Doping

Version: 0.3

Description: An implementation of the Abnormal Blood Profile Score (ABPS)

Imports: kernlab

License: GPL (>= 2)

Encoding: UTF-8

Author: Frédéric Schütz [aut, cre], Alix Zollinger [aut]

Maintainer: Frédéric Schütz <schutz@mathgen.ch>

How to specify dependencies

Dependencies allow you to specify which other packages your package need in order to do its job.

The package may even be empty and just specify dependencies, allowing users to use a single `library()` command to install a set of packages

Examples:

- Load all tidyverse packages using `library(tidyverse)`
- Load a single package that will in turn handle all dependencies required for an analysis

Specifying hard dependencies using Imports

You can specify hard dependencies using the `Imports` keyword in the `DESCRIPTION` file:

```
Imports: kernlab, ggplot2
```

Packages specified this way will be installed at the same time as your package.

Your package can not be installed if these dependencies are missing.

Specifying soft dependencies using Suggests

Packages indicated as "suggestions" can be used by your package, but they are not absolutely required in order to use it.

`Suggests: kernlab, ggplot2`

Packages listed in Suggests are not automatically installed along with your package; the package can be installed (and work) even if they are not present.

A typical use case

- Your analysis package absolutely requires packages that can perform the calculations (for example, `kernlab` for kernel methods)
- Without these, no output can be created
- These packages get **imported**.
- The package can also make pretty graphics with `ggplot2`
- But you can very well use the package and do something useful without the graphics
- The `ggplot2` package is only **suggested**.

Good R coding practices when writing packages

- Your R code should not change the user's environment.
- In particular, you should not call the `library()` or `require()` functions from within a package
- Instead, call the function in the following way:

`package::function()`

- Similarly, avoid settings parameters using functions such as `options()`, `par()` or `setwd()`.

How to check for the presence of a suggested package

Your package should always check for the presence of the suggested package before relying on it.

You can do this using the command:

```
requireNamespace(x, quietly=TRUE)
```

If a function requires the package

```
my_function <- function(a, b) {  
  if (!requireNamespace("my_package", quietly=TRUE)) {  
    stop("Package \"my_package\" is required.",  
        call.=FALSE)  
  }  
  ...  
}
```

If there is a fallback method

```
my_function <- function(a, b) {  
  if (requireNamespace("my_package", quietly=TRUE)) {  
    my_package::important_function()  
  } else {  
    other_function()  
  }  
}
```


Dependencies and suggestions can be versioned:

```
Imports: ggplot2 (>=2.2.0)
```

```
Suggests: ggvis (>=0.2)
```

Always depend on a given version **or higher**.

Dependencies on R versions using Depends

Dependency on a particular version of R can be specified using the Depends keyword:

```
Depends: R (>= 3.4.0)
```

Do **not** specify a version: Depends: R (==3.4.0)

"Depends" can also be used to specify packages; however, they will be automatically loaded and attached. "Imports" is usually the best choice.



Example: the devtools package

devtools: Tools to Make Developing R Packages Easier

Collection of package development tools.

Version: 2.4.5

Depends: R (≥ 3.0.2), [usethis](#) (≥ 2.1.6)

Imports: [cli](#) (≥ 3.3.0), [desc](#) (≥ 1.4.1), [ellipsis](#) (≥ 0.3.2), [fs](#) (≥ 1.5.2), [lifecycle](#) (≥ 1.0.1), [memoise](#) (≥ 2.0.1), [miniUI](#) (≥ 0.1.1.1), [pkgbuild](#) (≥ 1.3.1), [pkgdown](#) (≥ 2.0.6), [pkgload](#) (≥ 1.3.0), [profvis](#) (≥ 0.3.7), [rcmdcheck](#) (≥ 1.4.0), [remotes](#) (≥ 2.4.2), [rlang](#) (≥ 1.0.4), [roxygen2](#) (≥ 7.2.1), [rversions](#) (≥ 2.1.1), [sessioninfo](#) (≥ 1.2.2), stats, [testthat](#) (≥ 3.1.5), tools, [urlchecker](#) (≥ 1.0.1), utils, [withr](#) (≥ 2.5.0)

Suggests: [BiocManager](#) (≥ 1.30.18), [callr](#) (≥ 3.7.1), [covr](#) (≥ 3.5.1), [curl](#) (≥ 4.3.2), [digest](#) (≥ 0.6.29), [DT](#) (≥ 0.23), [foghorn](#) (≥ 1.4.2), [gh](#) (≥ 1.3.0), [gmailr](#) (≥ 1.0.1), [httr](#) (≥ 1.4.3), [knitr](#) (≥ 1.39), [lintr](#) (≥ 3.0.0), [MASS](#), [mockery](#) (≥ 0.4.3), [pingr](#) (≥ 2.0.1), [rhub](#) (≥ 1.1.1), [rmarkdown](#) (≥ 2.14), [rstudioapi](#) (≥ 0.13), [spelling](#) (≥ 2.2)

Published: 2022-10-11

Author: Hadley Wickham [aut], Jim Hester [aut], Winston Chang [aut], Jennifer Bryan  [aut, cre], RStudio [cph, fnd]

Maintainer: Jennifer Bryan <jenny at rstudio.com>

BugReports: <https://github.com/r-lib/devtools/issues>

License: [MIT](#) + file [LICENSE](#)

URL: <https://devtools.r-lib.org/>, <https://github.com/r-lib/devtools>

NeedsCompilation: no

Language: en-US

Materials: [README](#) [NEWS](#)

CRAN checks: [devtools results](#)

The (two) different kinds of R packages

Two kinds of R package

Type of package	Type of file	Hadley Wickam's nomenclature
	Directory	Source package
Source package	.tar.gz	Bundled package
Binary package	.zip, .tgz, ...	Binary package

A source package bundles the content of your package directory in one .tar.gz file (with a few changes)

It contains your entire package, and can be installed on any platform.

In the past, installing a source package could be difficult, depending on the version of R and the platform

Support for installing source packages has drastically increased over the years.

It now seems to be easy on all platforms (use version 3.4.4 or later)

Caveats about source packages

The receiving computer may need the development tools in order to install a source package: e.g. compiler, LaTeX, etc.

- Binary packages contains everything you need to install a package on a given architecture
- They contain libraries instead of source files
- R functions are stored in binary format
- There is no need for any additional tool

Each binary package is tailored for a single architecture:

- `.zip` for Windows
- `.tgz` for MacOS
- `$ARCH.tar.gz` for Linux; example:
`test_0.1_R_x86_64-pc-linux-gnu.tar.gz`

- Binary packages are useful if you distribute your package to people who do not have the development tools.
- For packages submitted to CRAN, binary versions will be automatically created for Mac and Windows.

On Linux, we do not usually use binary packages because:

- There are too many different architectures and distributions
- Most installations already have all the tools needed for building packages



How to create a binary package

Creating a binary package requires first compiling and installing the source package, and then making a bundle out of the installed files:

```
R CMD INSTALL --build SOURCE.tar.gz
```

Checking a package

R provides a **large** number of checks that can be applied to a package.

These will yields errors, warnings or notes about your package.

Checking a package

You should not distribute your package if it does not come clean from a check.

Checking a package

From the command line:

```
R CMD check sourcepackage.tar.gz
```

From Rstudio:

```
Build > Check Package
```

Check for CRAN

If you plan on uploading your package to CRAN, R can perform additional tests on it.

```
R CMD check --as-cran sourcepackage.tar.gz
```

You **must not** submit any package to CRAN if it does not come clean after this command.

Check the package that you have created before; try to understand any issue that the check command may report.

Adding documentation

Every function in a package should be documented.

Documentation is stored as .Rd files in the man/ subdirectory of your package.

The file format is similar to LaTeX.

Example of .Rd file for an add() command

```
\name{add}
\alias{add}
\title{Add together two
numbers}
\usage{
add(x, y)
}
\arguments{
  \item{x}{A number}

  \item{y}{A number}
}
```

```
\value{
The sum of \code{x} and
\code{y}
}
\description{
Add together two numbers
}
\examples{
add(1, 1)
add(10, 1)
}
```

Using roxygen2 for writing documentation

The `roxygen2` and `devtools` packages offer an easier way to write documentation.

The documentation is written in the form of **literate programming**: together with your code.

In practice, you add comments (in a specific format) to your R code.

They are then transformed into `.Rd` files

Example: Roxygen2 documentation

```
#' Add together two numbers.  
#'  
#' @param x A number.  
#' @param y A number.  
#' @return The sum of \code{x} and \code{y}.  
#' @examples  
#' add(1, 1)  
#' add(10, 1)  
add <- function(x, y) {  
  x + y  
}
```


Transforming the comments into .Rd files

To create the .Rd files: use the command

```
devtools::document()
```

(or use the relevant shortcuts in Rstudio:
CTRL/CMD – shift – D)

- All lines start with
'
in order to differentiate them from code and
regular comments
- The lines should form a single block before a
function
- Lines should not be over 80 characters long

- The first sentence (on the first) line will be the title of the help page
- The second paragraph will contain the introduction
- The next paragraphs will contain the details

You can create arbitrary sections using the @section tag:

```
#' @section Note:
```

```
#' The function has the same name, but a
```

```
#' different output, as the one provided in
```

```
#' the previous version of the package.
```

Most functions will have at least 3 sections defined by given tags:

- `@param NAME description`
to define the function parameters:
- `@return` to define the value(s) returned
- `@examples` to provide executable R code as an example of use of the function

Example

```

#' A function for calculating XYZ
#'
#' The \code{XYZ} function computes XYZ.
#'
#' @param X a vector or matrix containing the X data
#' @param Y a vector containing the Y data (not needed
#'         if the X parameter is a matrix)
#'
#' @return a vector containing the XYZ score(s).
#'
#' @examples
#' XYZ(1,1)

```

roxygen2 can also manage your NAMESPACE file for you:

```
#' ... documentation
#' @export
skewness <- function(a, na.rm=FALSE) {
  ...
}
```

Principle: it is easier to manage exports close to the function.

Adding data to a package

Several kinds of data in a package

- Data that will be called by the user using the `data()` command
- Internal data (e.g. precomputed data tables used by your method)
- Raw data used as example

- The dataset should be provided in the data/subdirectory
- They should be in .Rdata format, which you can obtain using the `save()` command.
- Typically:

```
mydata <- read.csv("mydata.csv")  
save(mydata, "data/mydata.Rdata")
```

Datasets are exported by default, so that you do not need to export them explicitly.

You can document data using roxygen2, by adding the documentation into the R/ directory (for example, in a data.R file)

Specify the name of the dataset at the end of the documentation (after the block):

```
#' Example of mydata
```

```
#' ...
```

```
#' End of description
```

```
"mydata"
```

Adding unit tests

Why add unit tests ?

Adding unit tests allows you to formalize the testing of your package.

It improves the quality of your package by making sure that at least some basic functionalities work as expected.

Most importantly, it allows you to make sure that what worked once continue to work after you make changes to the package (you don't introduce new bugs)

You can add a framework for tests to your package by running:

```
devtools::use_testthat()
```

You can then run tests using

```
devtools::test()
```

Or by running a check on your package.

- A "Suggests: testthat" in the DESCRIPTION file
- A tests/ directory
- A tests/testthat.R script that will run all your tests during package check
- A tests/testthat/ subdirectory, containing
 - Files test*.R containing the actual tests

Example of testthat.R file

```
library(testthat)
```

```
library(packagename)
```

```
test_check("packagename")
```

What is a test file ?

```
context("The addition function")
library(mylibrarytotest)

test_that("The addition works", {
  expect_equal(add(1,4), 5)
  expect_equal(add(1,-1), 0)
  expect_equal(add(0,0), 0)
})
```

What is an EXPECTation ?

Expectations (functions starting with `expect_`) allow you to specify an R command, as well as the result you expect from it:

```
expect_equal(a, b)
```

```
[allow for tolerance]
```

```
expect_identical(a, b)
```

```
expect_error(add("a", "b"))
```

Publishing your package

Publishing a package

- On CRAN
 - Special checks
 - Licence
 - Attention to details !
-
- On a website
 - On Github